ФБ-51мн Войтенко Дмитро

ЛАБОРАТОРНА РОБОТА №1

Тема: Вибір та реалізація базових фреймворків та бібліотек

Мета: дослідження сучасних програмних бібліотек реалізації криптографічних механізмів, критеріїв їх вибору для різних програмно-апаратних платформ та отримання практичних навичок коректного використання криптографічних API.

1. Дослідження принципів реалізації криптографічних механізмів

- **Чому не рекомендується самостійно реалізовувати алгоритми?** Самостійна реалізація може працювати математично правильно, але містити критичні вразливості на рівні коду: витоки через час виконання (timing attacks), помилки перевірки вхідних даних, проблеми з очищенням пам'яті. Готові бібліотеки перевірені криптоаналітиками та спільнотою.
- **Основні групи примітивів:** Симетричне шифрування (AEAD), асиметричне шифрування, алгоритми цифрового підпису, геш-функції, механізми узгодження ключів та функції формування ключів (KDF).
- **Різниця між примітивом та протоколом:** Примітив — це базова математична операція (наприклад, AES шифрує блок, SHA-256 робить геш). Протокол — це набір правил взаємодії систем, що використовує примітиви для досягнення мети (наприклад, TLS або SSH).
- **Що таке CSPRNG:** Це криптографічно стійкий генератор псевдовипадкових чисел. Його наступне значення неможливо обчислити, навіть знаючи попередні. Звичайні генератори (PRNG) призначені для ігор та статистики — їх результати можна передбачити, що призведе до компрометації згенерованих ключів.
- **Різниця між low-level та high-level API:** High-level API (високорівневі) самостійно обирають безпечні налаштування (режими, padding) і приховують складність. Low-level API (низькорівневі) вимагають від розробника самостійно збирати алгоритм з деталей. Ризик: розробник

може обрати небезпечний режим (наприклад, ECB) або забути про автентифікацію шифротексту.

- **Constant-time implementation:** Це реалізація алгоритму, час виконання якої не залежить від секретних даних (ключів або паролів). Вона захищає систему від таймінг-атак (атак побічними каналами), коли хакер вираховує ключ, вимірюючи мілісекунди, які витрачає сервер на відповідь.
- **Що таке FIPS 140-3:** Це державний стандарт США, що визначає вимоги безпеки для криптографічних модулів (як програмних, так і апаратних). Сертифікація важлива для розробки державних, банківських, військових або медичних систем, де використання несертифікованої криптографії заборонено законом.
- **Арифметика багаторозрядних чисел для RSA:** RSA працює з числами довжиною 2048 або 4096 біт, які не поміщаються у стандартні типи даних процесора (32/64 біт). Окрім швидкодії, до цієї арифметики висуваються вимоги стійкості до атак побічними каналами (constant-time) та безпечного очищення оперативної пам'яті після виконання розрахунків.

2. Порівняння криптографічних бібліотек

Критерій	**cryptography (Python)**	**PyCryptodome (Python)**
Симетричне шифрування та AEAD	AES-GCM, ChaCha20-Poly1305	AES (GCM, CCM, OCB), ChaCha20
Асиметрична криптографія та підпис	RSA, X25519, ECDSA, Ed25519	RSA, ECDH, ECDSA, EdDSA

Критерій	**cryptography (Python)**	**PyCryptodome (Python)**
Високорівневе API	Є (наприклад, модуль Fernet)	Відсутнє, вимагає ручного налаштування
Апаратне прискорення	Залежить від збірки OpenSSL (backend)	Є підтримка AES-NI на x86
Ліцензія	Apache 2.0 / BSD	BSD / Public Domain

Обґрунтування вибору: Для подальшої роботи обрано бібліотеку cryptography. Вона є сучаснішим стандартом у Python-спільноті, містить зручні високорівневі абстракції (recipes) та надійно реалізує перевірку цілісності даних при автентифікованому шифруванні (AEAD).

3. Практичне використання криптографічної бібліотеки

```
--- 3.1 Симетричне автентифіковане шифрування ---
Алгоритм: ChaCha20-Poly1305
Початкове повідомлення: Secret data for scenario
Ключ: 256 біт (секретне значення не виводиться)
Nonce (12 байт): 16b97ab05d39f57fe9a5daec
AAD: metadata: user=123
Шифротекст (без тегу): 49e6ac76447ceba2a7f6635ec1307f8e56542f85ab5c1ee1
Authentication Tag (16 байт): d4d6970f2f3ad2134b03f498e3ed7f34
Розшифроване повідомлення: Secret data for scenario
Статус: УСПІШНО

Спроба розшифрувати підроблений шифротекст: InvalidTag
Статус: Змінений шифротекст успішно відхилено

--- 3.2 Цифровий підпис ---
Алгоритм: ECDSA (SECP256R1)
Початкове повідомлення: Document to sign: Contract #45
Відкритий ключ (hex): 032157a9f5d49ec48d8dafc21b323caa7d5ff04cb6ab2c7e7360b0591668491ccf
Підпис (hex): 3045022100951d6a34dff9c19ff008b82c740a1e21d9e2f895d20abc659c5e0d...
Перевірка підпису оригінального повідомлення: УСПІШНО

Змінене повідомлення: Document to sign: Contract #99
Перевірка підпису зміненого повідомлення: InvalidSignature
Статус: Підроблений документ успішно відхилено
```

Пояснення призначення параметрів:

- **Ключ:** Секретний параметр для симетричного шифрування або підписання.
- **Nonce / IV:** Одноразове унікальне значення (12 байт). **Вимоги:** Nonce алгоритму ChaCha20 ніколи не повинен повторюватися для одного ключа. Наслідок порушення: повна компрометація ключа та розшифрування всіх повідомлень.
- **Authentication tag (тег автентифікації):** Гарантує цілісність шифротексту. Якщо дані були модифіковані (як у контрольному прикладі), алгоритм відхиляє розшифрування з помилкою InvalidTag.

4. Вибір криптографічної бібліотеки для різних IT-систем

- **Сценарій А (Backend-сервіс):** Висока швидкодія (підтримка апаратного прискорення процесора), безпечна паралельна робота, можливість інтеграції з зовнішніми KMS.
- **Сценарій Б (Мобільний застосунок):** Інтеграція з нативними апаратними сховищами ключів пристрою (Android Keystore / iOS Secure Enclave) для заборони експорту секретів.
- **Сценарій В (Сервіс цифрового підпису):** Робота з інтерфейсом PKCS#11 для делегування операцій підписання до Hardware Security Module (HSM).
- **Сценарій Г (IoT-пристрій):** Низьке споживання оперативної пам'яті (RAM) та ініціалізація від апаратного генератора ентропії мікроконтролера.

5. Індивідуальне завдання (Варіант В: Сервіс цифрового підпису)

1. **Вимоги до ПЗ:** Абсолютний захист закритого ключа від експорту/копіювання; суворий аудит операцій підписання; неможливість підміни гешу документа.

2. **Обрана бібліотека/API:** cryptography для високорівневої обробки та гешування + інтерфейс PKCS#11 для виклику апаратного модуля.
3. **Визначені функції:** Алгоритми підпису (ECDSA, RSA-PSS), геш-функції (SHA-256/384).
4. **Додаткові механізми:** Обов'язкове використання мережевих або локальних HSM (Hardware Security Module) типу "Гряда". Ключ генерується з атрибутом non-exportable.
5. **Обґрунтування:** Використання ізольованого HSM гарантує, що навіть у разі повного компрометування серверної інфраструктури (root-доступу хакера), закритий ключ не залишить межі фізичного пристрою.
6. **Ризики:** Повторне використання значення k (nonce) при формуванні підпису ECDSA повністю розкриває закритий ключ; відсутність перевірки автентичності відкритого ключа другої сторони.

Висновки

Під час виконання лабораторної роботи було досліджено сучасні криптографічні бібліотеки та примітиви. На практиці доведено, що використання високорівневих API мінімізує ризик фатальних помилок розробника. За допомогою бібліотеки cryptography успішно реалізовано алгоритм автентифікованого шифрування ChaCha20-Poly1305 та алгоритм цифрового підпису ECDSA. Перевірки довели, що алгоритми ефективно блокують будь-які спроби модифікації даних та підробки підписів (виключення InvalidTag та InvalidSignature). Для індивідуального завдання (Сервіс цифрового підпису) розроблено концепцію захисту ключової інформації за допомогою апаратних модулів HSM, що виключає можливість викрадення закритого ключа.

ФБ-51мн Войтенко Дмитро